

Database Design Entity-Relationship (ER) Modeling: Relationships

COSC 304 – Introduction to Database Systems



★ ER Modeling

Relationship Multiplicities



Relationship multiplicities specify how many instances of one entity type may be associated with an instance of another entity type. They model real-world constraints.

A multiplicity is expressed as a **minimum..maximum** range.

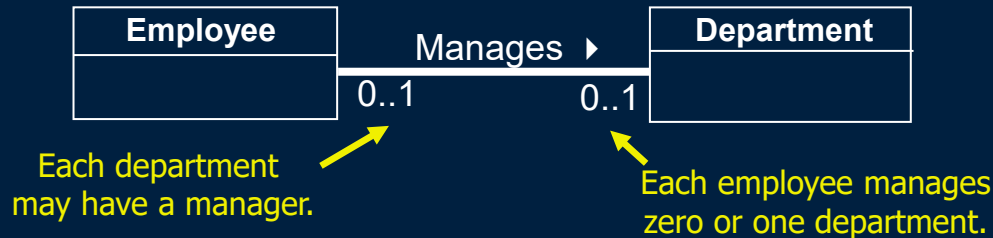
The **maximum cardinality** of a binary relationship gives three common patterns:

- one-to-one (1:1)
- one-to-many (1:* or 1:N)
- many-to-many (*:* or N:M)

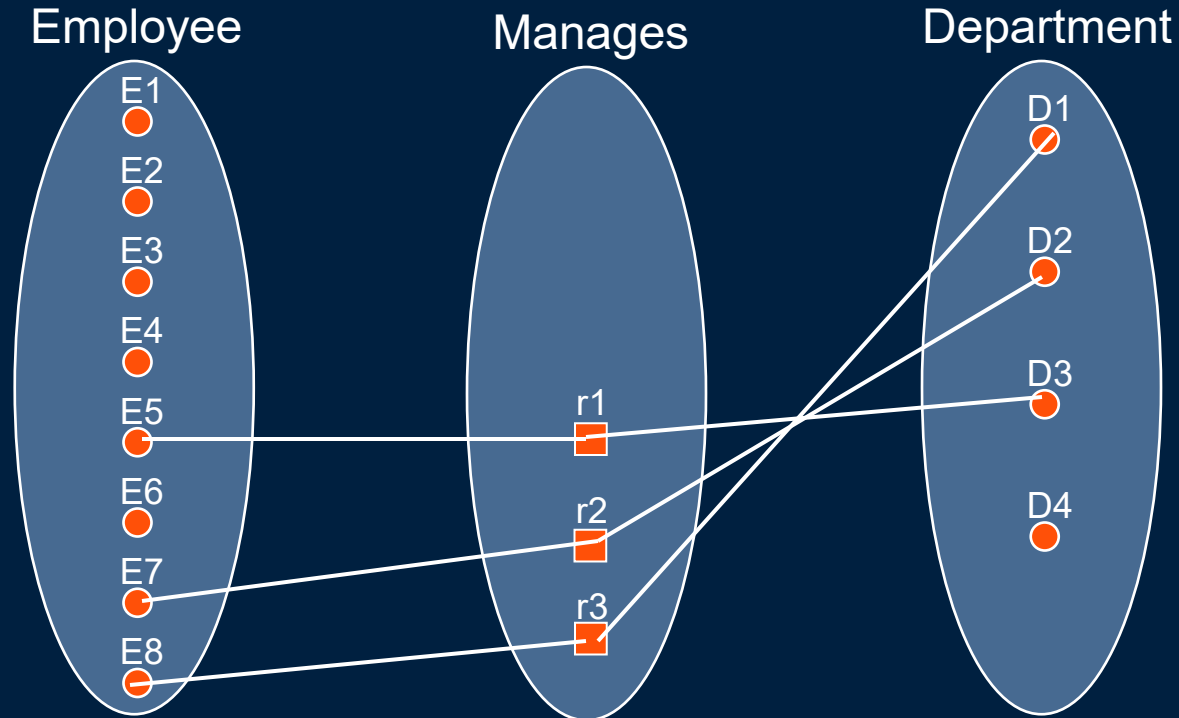
One-to-One Relationships

In a one-to-one relationship, each instance of entity type E1 can be associated with **at most one** instance of entity type E2 and vice versa.

Example: A department may have only one manager, and a manager may manage only one department.



One-to-One Relationship Example

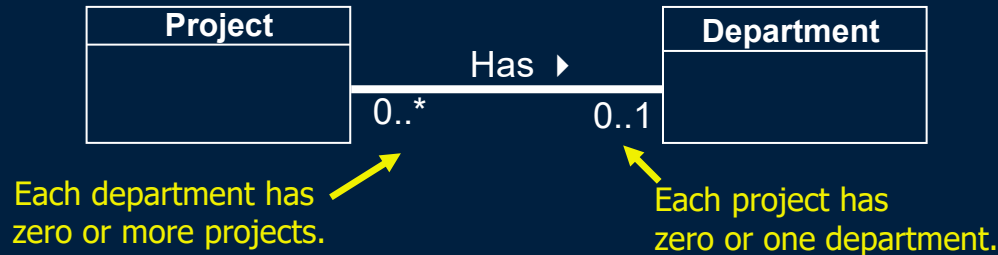


Relationship explanation: A department has at most one manager. A manager (employee) may manage at most one department.

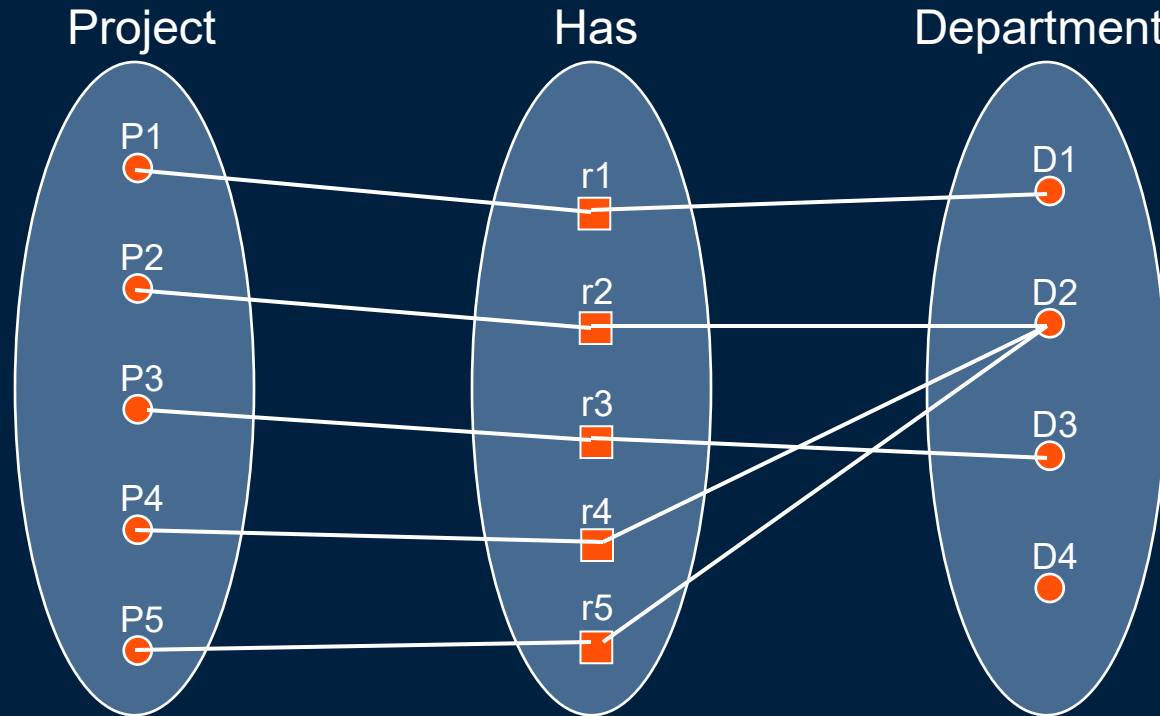
One-to-Many Relationships

In a one-to-many relationship, an instance of entity type E1 may be associated with **multiple** instances of entity type E2. However, E2 can only be associated with **at most one** instance of E1.

Example: A department may have multiple projects, but a project may have only one department.



One-to-Many Relationship Example

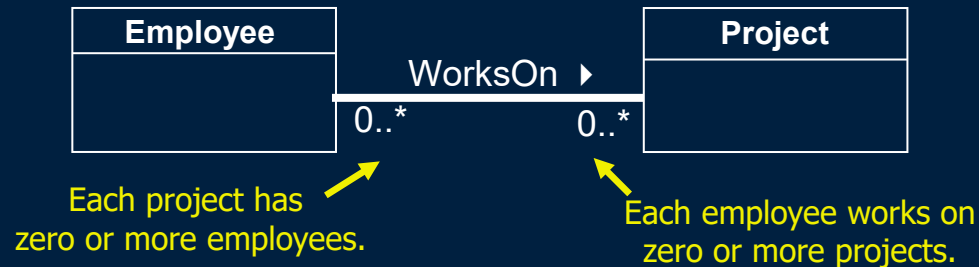


Relationship: One Department may have many Projects. Each Project belongs to at most one Department.

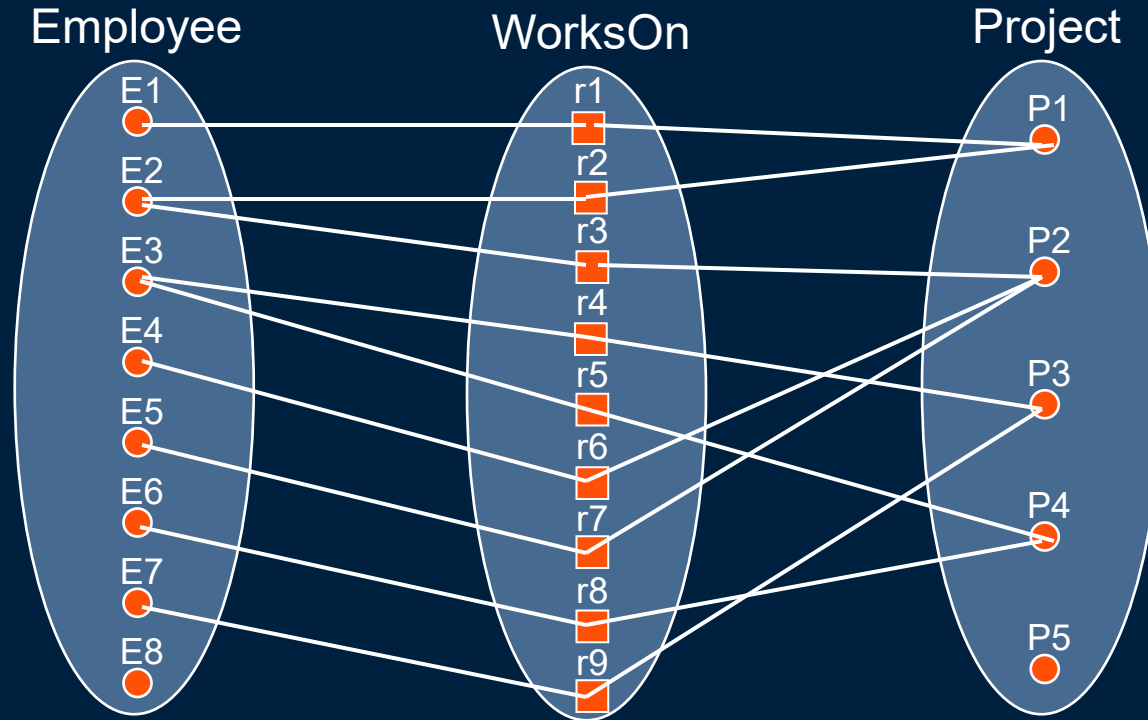
Many-to-Many Relationships

In a many-to-many relationship, an instance of entity type E1 may be associated with **multiple** instances of entity type E2 and vice versa.

Example: An employee may work on multiple projects, and a project may have multiple employees working on it.



Many-to-Many Relationship Example



Participation Constraints

Cardinality is the *maximum* number of relationship instances for an entity participating in a relationship type.

Participation is the *minimum* number of relationship instances for an entity participating in a relationship type.

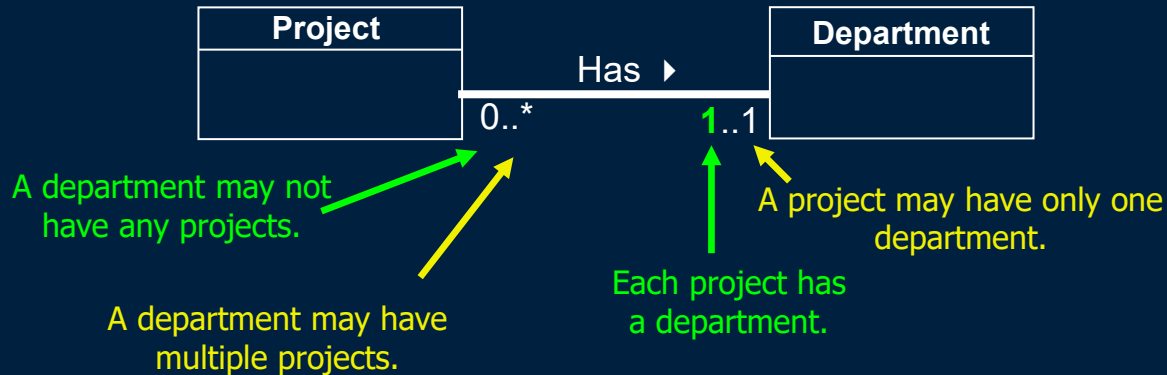
- Participation can be *optional (zero)* or *mandatory (1 or more)*.

If participation is mandatory (also called *total* participation), every entity instance must participate in at least one instance of the relationship. This is an existence dependency.

- Existence dependency does not necessarily imply a weak entity. A weak entity also lacks a complete key of its own.

Participation Constraints Example

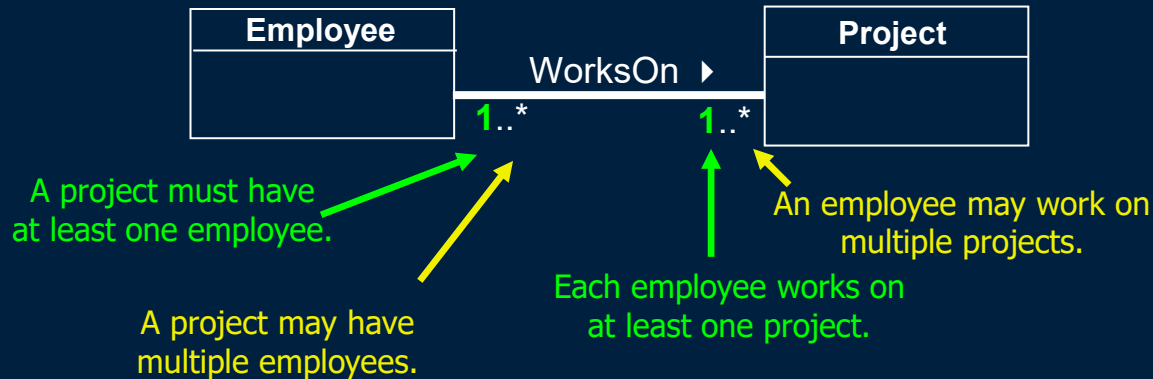
Example: Each project must be associated with exactly one department, while a department may have zero or more projects.



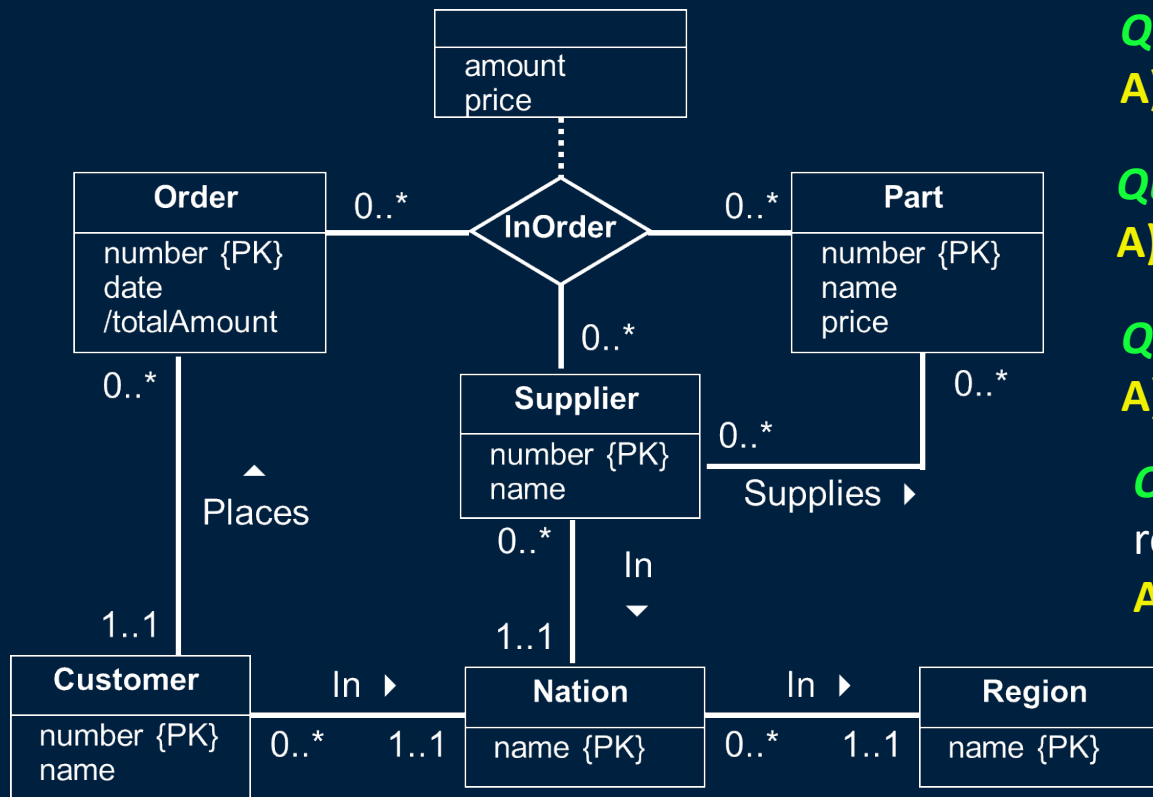
Note: Every project must participate in the relationship (mandatory).

Participation Constraints Example 2

Example: A project must have one or more employees, and an employee must work on one or more projects.



TPC-H ER Diagram Questions



Question 1: How many entity types?

A) 5 B) 6 C) 7 D) 8 E) 9

Question 2: How many relationships?

A) 4 B) 5 C) 6 D) 7 E) 8

Question 3: How many primary keys?

A) 4 B) 5 C) 6 D) 7 E) 8

Question 4: How many binary 1-N relationships?

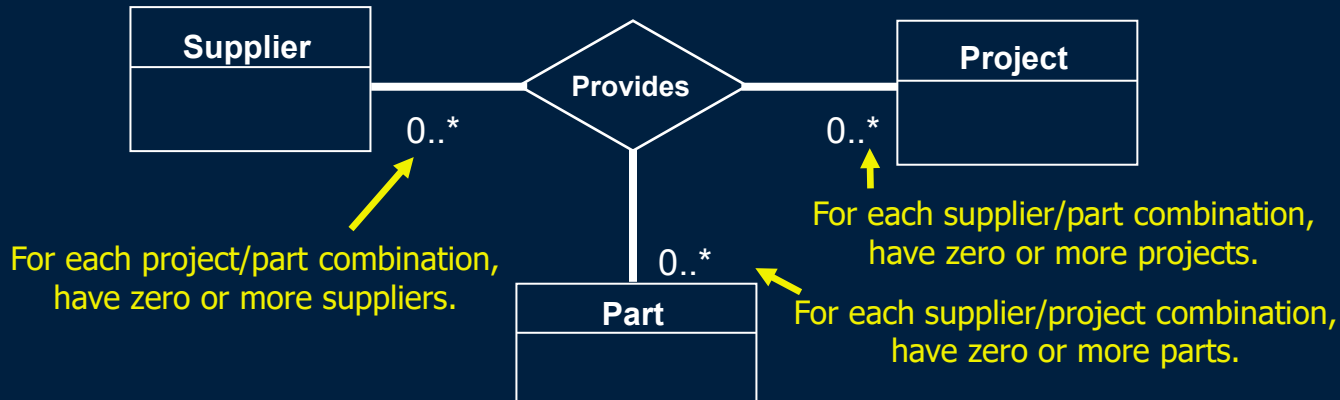
A) 4 B) 5 C) 6 D) 7 E) 8

Question 5: How many foreign keys? A) 3 B) 4 C) 5 D) 9 E) 0

Multiplicity of Non-Binary Relationships

For an n -ary relationship, the multiplicity of one entity type is interpreted while the other $n-1$ entity instances are fixed.

Example: `Provides(Supplier, Part, Project)` indicates which suppliers provide which parts for which projects. Each multiplicity is interpreted while fixing the other two participants.



Relationship Cardinality Question

Question: How many of the following statements are **true**?



- 1) An entity of type A must be related to an entity of type B.
- 2) An entity of type A may be related to more than one entity B.
- 3) There is a 1-to-many relationship from A to B.
- 4) An entity of type B must be related to an entity of type A.
- 5) An entity of type B must be related to more than one entity of type A.

A) 0 B) 1 C) 2 D) 3 E) 4

Multiplicity Practice Question

Consider the university database developed before. Write multiplicities into the ER diagram given that:

- A department must offer at least one course.
- Courses are offered by only one department.
- A course may have multiple sections, but always has at least one section.
- A student may enroll in multiple course sections (but does not have to). A section may have multiple students enroll.
- A professor may be in multiple departments (at least 1), and a department must have at least one professor.
- A section is taught by at least one professor, but may be taught by more than one. A professor does not have to teach.

Strong and Weak Entity Types

A **strong entity type** has a primary key composed of its own attributes that uniquely identifies its instances.

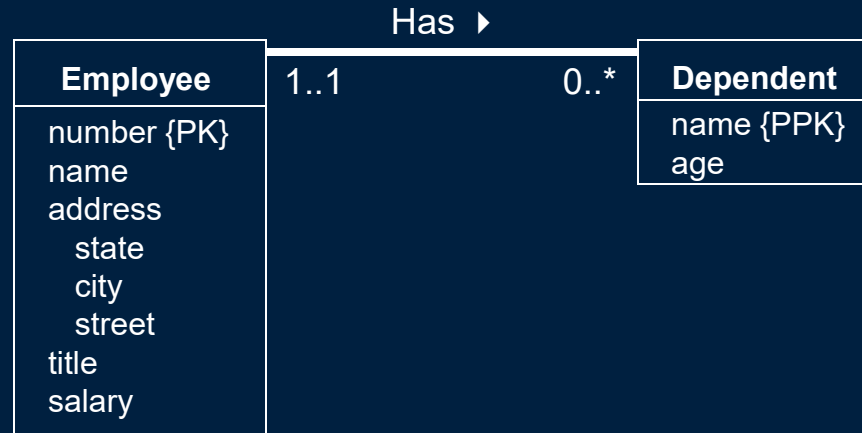
A **weak entity type** does not have a complete key of its own and is identified through an owner entity together with a partial key.

- A weak entity has an existence dependency on its identifying relationship.

A common example of strong and weak entity types are employees and their dependents:

- An employee is a strong entity because it has an employee number to identify its instances.
- A dependent (child) is a weak entity because its own attributes do not uniquely identify it. They are identified by the parent's employee number and their name.

Weak Entities in UML



Strong and Weak Entity Question

Question: How many of the following statements are **true**?

- 1) A weak entity has its own primary key.
- 2) A strong entity has its own primary key.
- 3) A weak entity must be associated (identified) by a strong entity.
- 4) A weak entity can have a relationship with another entity besides its identifying strong entity.
- 5) The attribute(s) of a weak entity used to identify it with its associated strong entity are noted as { PPK } in the UML model.

A) 0 B) 1 C) 2 D) 3 E) 4

ER Modeling – Entity, Relationship, or Attribute?

Basic challenge is when to model a concept as an entity, a relationship, or an attribute. In general:

- Entities are nouns.
 - You should be able to identify a set of key attributes for an entity.
- Attributes are properties and may be nouns or adjectives.
 - Model a concept as an attribute when it is a property of a single entity or relationship and does not need to be identified independently.
 - Model a concept as an entity when it has independent identity, attributes of its own, or participates in relationships.
- Prefer binary relationships when they naturally represent the domain, but use n -ary relationships when the association genuinely depends on all participating entity types. Note that non-binary relationships can be modeled as an entity instead.

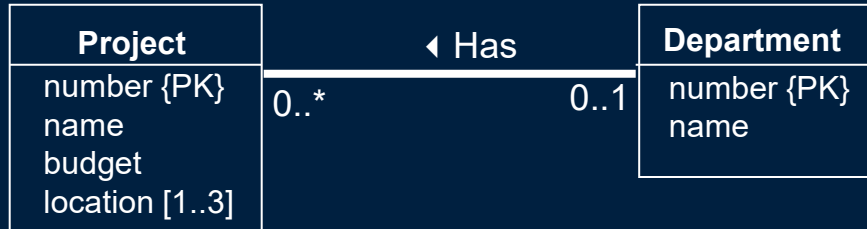
Use weak entities when an entity is naturally identified through an owning entity. In implementations, surrogate identifiers are also commonly introduced.

Good Design Practices

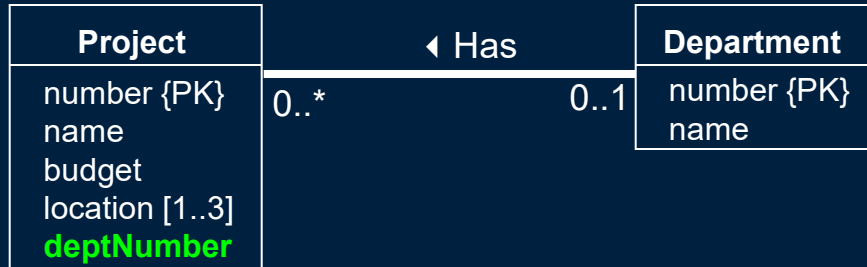
Avoiding Redundancy Example



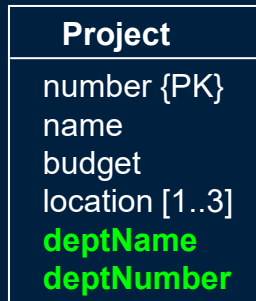
Good:



Wrong:



Bad:



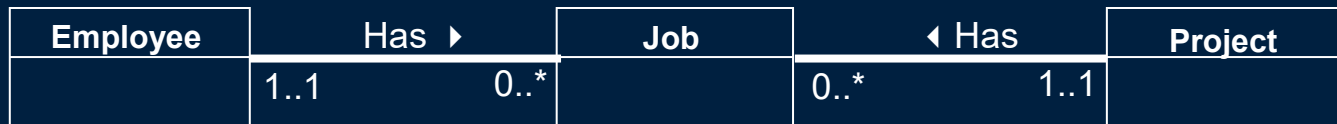
Representing a Many-to-Many Relationship as an Entity

A many-to-many relationship can be converted into one entity with two 1:N relationships between the new entity and the original entities participating in the relationship.

Original:

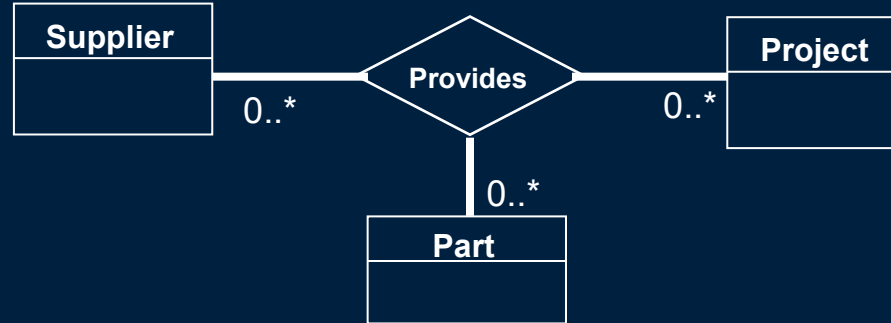


Associative
Entity:

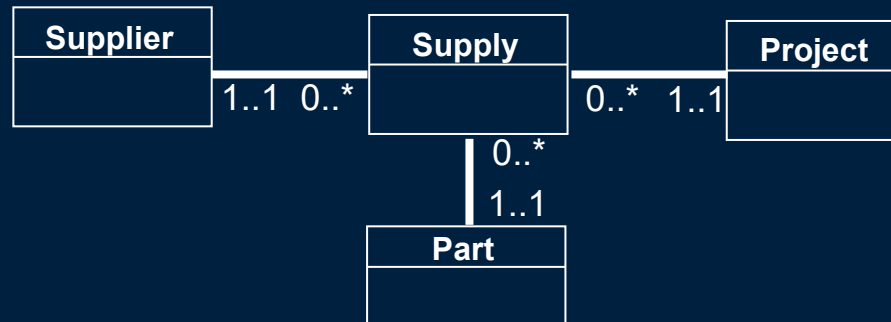


Representing a Higher-Degree Relationship as an Entity

Original:



Associative entity:

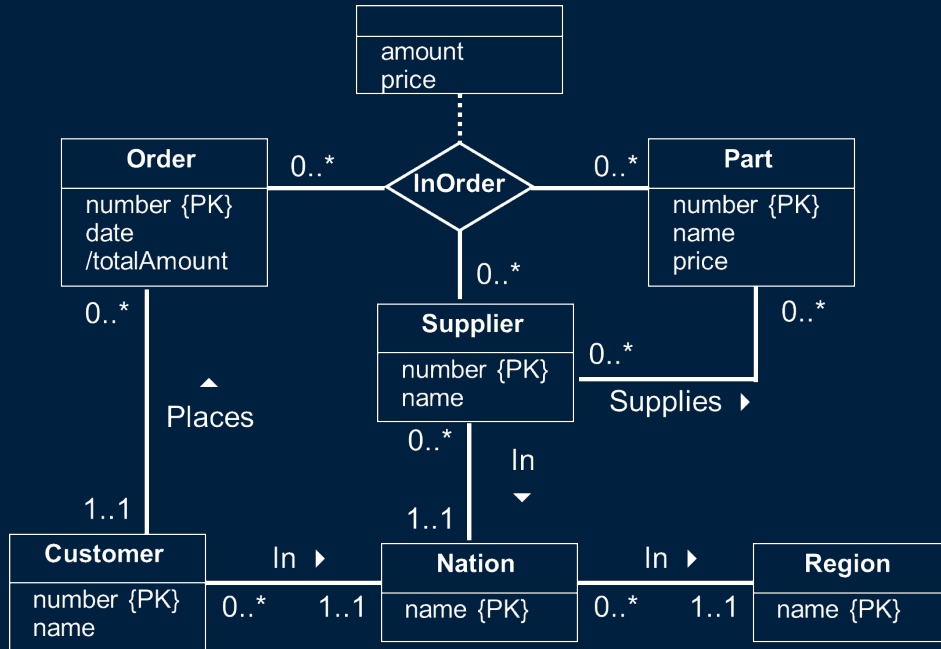


ER Design Example

Simplified TPC-H ER Design



- Each order has a numeric key, a customer, a date, a total order amount, and a list of parts.
- A part in an order has an amount and a price paid and is supplied by a certain supplier.
- Each part has a numeric key, a name, and a price and may be supplied by multiple suppliers.
- A supplier has a key and a name and may supply multiple parts.
- A customer has a key and a name.
- Each supplier and customer is located in a nation.
- Each nation is located in a region (continent).



ER Design Question #2

Construct a fish store database where:

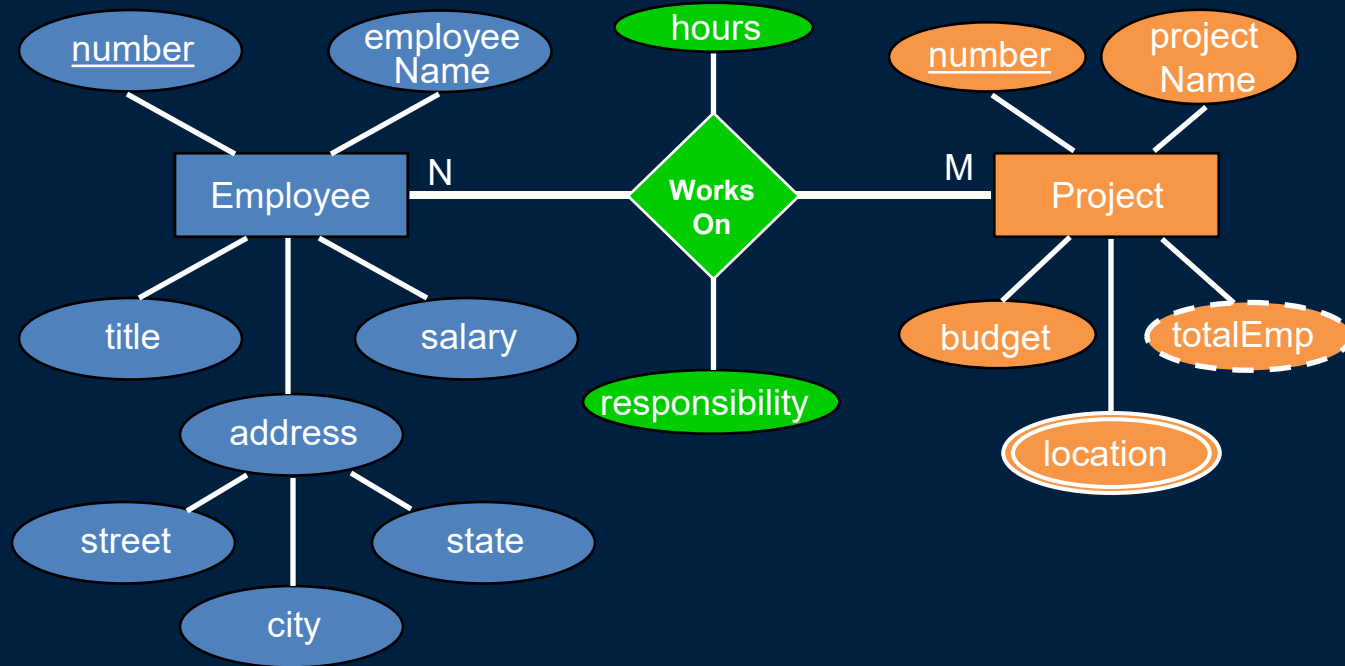
- A fish store maintains a number of aquaria (tanks), each with a number, name, volume and color.
- Each tank contains a number of fish, each with an id, name, color, and weight.
- Each fish is of a particular species, which has an id, name, and preferred food.
- Each individual fish has a number of events in its life, involving a date and a note relating to the event. Assume a fish has at most one event on a given date.

ER Design Question #3

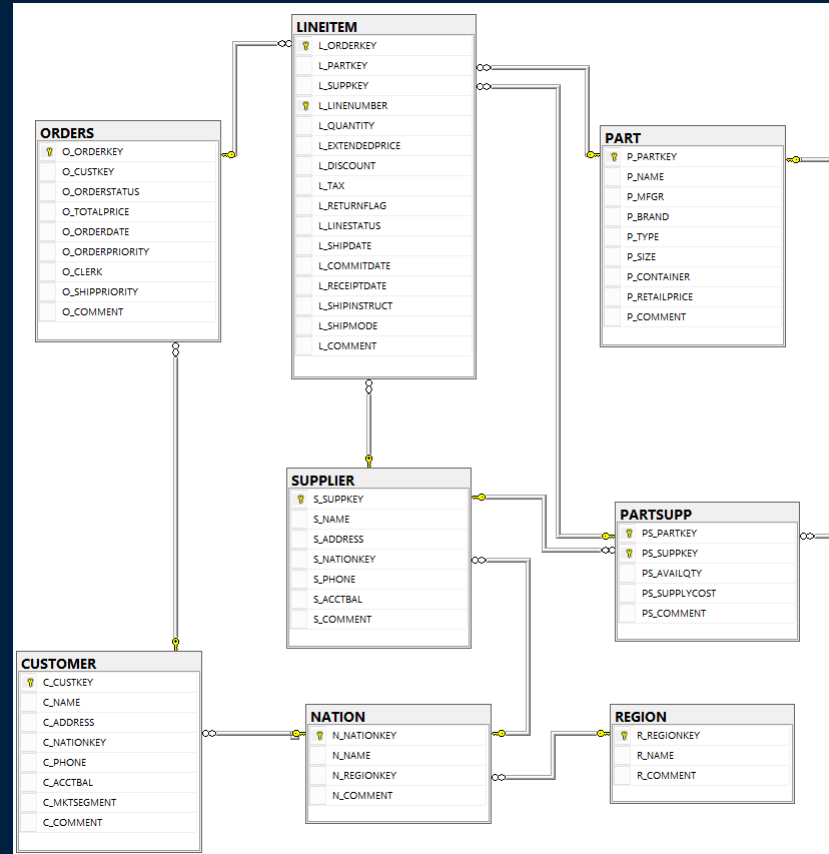
Construct an invoice database where:

- An invoice is written by a sales representative for a single customer and has a unique ID. An invoice has a date and total amount and has multiple detail lines, each containing a product, price and quantity.
- Each sales representative has a unique name and can write many invoices, but any invoice is written by a single representative.
- Each customer has a unique id, name, and address and can request many invoices.
- Products have descriptions and weights and are supplied by vendors. Each product has a unique name for a particular vendor. A product is supplied by only one vendor. A product appears at most once on an invoice.
- A vendor has an id and an address.

ER Model – Chen Historical Notation



Other Notations – Logical Relational Diagram



Conclusion

Conceptual design is performed at a high level of abstraction involving entities, relationships, and attributes.

- An entity type is a group of entities with the same properties.
 - Entities may be strong (have a key of their own) or weak (identified through an owner entity and a partial key).
- A relationship type represents associations among entity types.
 - A relationship may involve two or more entities and may be recursive.
 - A relationship has two types of constraints:
 - Participation - minimum # of times an entity must be involved in relationship
 - Cardinality - maximum # of times an entity can be involved in relationship
 - Common relationship multiplicities are: 1:1, 1:*, *:.*.
- Attributes are properties of entities or relationships.

Good design requires practice.

Objectives

- Define and interpret cardinality, participation, and multiplicity constraints in an ER diagram.
- Interpret and construct 1:1, 1:N, N:M, and n -ary relationships.
- Explain the difference between strong and weak entity types and use partial primary keys.
- Decide when a concept should be modeled as an entity, relationship, or attribute.
- Recognize when a relationship can be represented using an associative entity.



Construct an ER model from a domain description using UML-style notation.



THE UNIVERSITY OF BRITISH COLUMBIA

